“Security Considerations for Artificial Intelligence Agents”
A response to questions in NIST-2025-0035

Background:

This document uses the NIST Request for Information on “Security Considerations for Artificial Intelligence” as a framework to discuss AI agent security. The questions are sourced from the Federal Register post by that title, with the number listed above.

Focus / terminology:

Although most widely used AI agent systems are Transformer architecture LLMs (proposed in 2017),[50] AI textbooks published as early as 1995[49] discuss AI based agents.

The NIST RFI did not explicitly state its focus as LLM-based AI agents but the context and references it included support this interpretation. This document uses the terms “AI agent” and “agent” as shorthand for LLM-based AI agents.

Responses:

1. Security Threats, Risks, and Vulnerabilities Affecting AI Agent Systems

Question 1(a)

1.(a) What are the unique security threats, risks, or vulnerabilities currently affecting AI agent systems, distinct from those affecting traditional software systems?

The combination of natural language interfaces, learning components, autonomy, and tool integration create threats, vulnerabilities, and potential impacts beyond the scope of classical software threats.

1. Prompt injections:
   Prompt injection attacks (both direct and indirect) are enabled by the following characteristics of models used in most AI agent systems:

   Unbounded natural-language instruction following
   - instructions expressed in the same natural-language form as untrusted content

   Blended control and data channel
   - a single shared input stream contains system instruction, user instructions, and untrusted content.

   Weak instruction-data boundary enforcement
   - no strict syntactic boundaries between instructions and untrusted data
   - the model relies on learned semantic cues to distinguish system instructions from user instructions or data.

   Weak focus
   - LLMs are easily distracted from their original instructions by prompt injections. [48]

   In 2025, 0-click "hijacking" exploits were found impacting organizations where M365 Copilot ("*EchoLeak*"[8]) or OpenAI ChatGPT ("*ShadowLeak*"[11]) agents had access to user mailboxes. Both exploits resulted in the agent successfully exfiltrating data after viewing an email whose text included indirect prompt injection.

2. Cascading prompts in multi-agent systems:
   These include cascading prompt injections, cascading harmful misinterpretations of legitimate instructions, or cascading errors which propagate and amplify from agent to agent.[28][47]

3. Data poisoning, backdoors, evasion:
   Adversarial ML risks amplified in AI agent systems by system-level risks (tool supply chain, authentication misuse). [7][2][5]

   These can remain hidden until a particular trigger occurs and have been shown to survive retraining the model. [56]

4. Misaligned goals:
   AI agents can choose unintended and harmful methods to complete an assigned goal and can pursue goals they were not assigned ("excessive agency").

A Replit.com AI coding agent deleted a Replit client company's production database. The agent then added application code to hide the deletion by simulating the existence of the database. [52]

The AI agent in the Replit incident wrote code to fake the existence of the database it deleted – actively attempting to hide its error. AI agents have also been observed to attempt to hide misalignment when being tested ("alignment faking"). [57]

Anthropic researchers reported testing of *"16 major models from Anthropic, OpenAI, Google, Meta, xAI, and other developers … found consistent misaligned behavior: models that would normally refuse harmful requests sometimes chose to* ***blackmail****,* ***assist with corporate espionage****, and even take some more extreme actions, when these behaviors were necessary to pursue their goals."*

Researchers from Stanford, Harvard, MIT, Carnegie Mellon, and other institutions conducted a red-teaming study of AI agents, and observed *"unauthorized compliance with non-owners, disclosure of sensitive information, execution of destructive system-level actions, denial-of-service conditions, uncontrolled resource consumption, identity spoofing vulnerabilities, cross-agent propagation of unsafe practices, and partial system takeover".* [51] Anthropic researchers found similar results across models from all major providers. [53]

5. Other deceptive behaviors:
   Strategic deception, "sandbagging", sycophancy, unfaithful reasoning

   This threat type also includes "banal deception" – the common "hallucinations" familiar to AI users. Like other threat types, the potential impact of these is far greater when combined with overconfidence in AI agents.

6. Human overconfidence in AI agents:
   Overconfidence in AI agents, a pervasive risk amplifier, leads to blind spots (lack of logging and monitoring), inattention (ignored logging and monitoring), de-prioritization (over-trust in traditional security), and lack of preparation (inadequate incident response and disaster recovery planning).

   Overconfidence in agents:
   - Unquestioning acceptance of AI agent outputs
   - Misapplication of AI agent technology
   - Lack of focus on AI agent security
   - Unfounded trust in pre-existing security controls
   - Assumptions of "good" behavior
   - Over-permissioning AI agents
   - Overprovisioning functionality to AI agents
   - Inadequate preparation for adverse outcomes

   Overconfidence in AI agents amplifies the risks (probability and potential impact) of all AI agent incident types. As AI agents are deployed in critical national security contexts, the potential risks of overconfidence merit serious consideration.

Question 1(b)

1.(b). How do security threats, risks, or vulnerabilities vary by model capability, agent scaffold software, tool use, deployment method (including internal vs. external deployment), hosting context (including components on premises, in the cloud, or at the edge), use case, and otherwise?

If AI agent tool capabilities are scoped to the least required functionality for the use case with least privileged identities scoped by separation of responsibilities, risks may be estimated and controls applied effectively. Note that some risks discussed in this and other answers can be mitigated but not eliminated in AI agent systems.

Capability, tools, scaffolding:
Risk increases directly with increases in capability (tools, model reasoning), persistence (long context or persistence using caching, memory). Identities used and their privilege level matters, as well.

Some AI service providers have implemented tools directly in prompts, creating risks for organizations using these models.

Example from personal experience:
The current versions of Google Gemini models include the "URL Context" feature. If a prompt includes a web URL, Google directly retrieves data from that URL and adds the data to the context. Organizations using Google Gemini in agents could not disable this feature, including use of Google Gemini agents in GitHub Copilot.

Use of agents with persistent memory includes risk of cross-session persistence of manipulations or own-goal-seeking not present in stateless single-turn agents. Least-privilege "read-only" assistants (typically RAG) are easier to secure than AI agent systems that write data, interact with, or administer systems.

Use of MCP (Model Context Protocol) increases development velocity but includes risk of threats hidden in tool definitions or metadata, or in agent and tool interactions. A VirusTotal survey found > 8% of 18,000 MCP server projects scanned appeared intentionally malicious.[17]

The design and implementation of tools and scaffolding, and choice and privilege of identities used in scaffolding directly constrain or increase the probability and potential impact of agent incidents.

Deployment method / hosting context:
Externally exposed agents attract more outsider attacks such as prompt injections and unauthorized access attempts. SaaS deployments may face multi-tenancy and supply-chain vulnerabilities.

Internal deployments may be partially safeguarded by existing enterprise security but must assume insider threats and potential misconfigurations.[8] In some organizations, over-trust in traditional perimeter defenses leads to deployments left exposed to arbitrary internal connections.

Use Case:
Like other software, use of AI agents in critical infrastructure, healthcare, or government includes greater risk of safety and privacy threats.[8]

Question 1(c)

To what extent are security threats, risks, or vulnerabilities affecting AI agent systems creating barriers to wider adoption or use of AI agent systems?

High-profile security incidents may make some companies wary of wider adoption of AI agent systems.

However, possible barriers to wider adoption should currently be a lesser concern than deployments rushed into production without due care for AI agent security. Unfortunately, many such deployments are in process or already in production in many organizations.

For Gravitee's "*State of AI Agent Security 2026*" [17] report, polling firm Opinion Matters surveyed 750 CTOs and tech VPs.

AI Projects in the past year:

Pilot/testing or production	80.9%
Using AI agents	80.3%

Percentage of agents approved by IT/Security

All agents approved	**14.4%**
Most agents approved	43.1%
Some agents approved	34.3%
"Hardly any" approved	8.3%

AI agents actively monitored and secured: 47%

Organizations reporting AI agent security or privacy incidents in the past year

Suspected incidents	29%
Confirmed incidents	**59%**

Executives in these organizations

Confidence their policies could protect against misuse and unauthorized agent actions

"somewhat confident"	45.6%
"***very confident***"	**36.4%**

The survey results indicate AI agent projects driven to production without adequate security, with overconfidence even after security or privacy incidents have already occurred.

Question 1(d)

How have these threats, risks, or vulnerabilities changed over time? How are they likely to evolve in the future?

The focus of threats has shifted from simple issues affecting AI models, such as bias and hallucination, and attacks on training data, to vulnerabilities involving tool use, over-privilege, and indirect prompt injection through varied, arbitrary data types and sources.

Prompt injection attacks have grown from simple efforts to taint output to multi-step attacks through agents to exfiltrate data, such as the EchoLeak[8] and ShadowLeak[11] attacks.

Risks from agents themselves, without an external actor, are also now more obvious (e.g., Replit agent destruction of a customer database).

Agent security threats have increased dramatically as model capabilities, autonomous capabilities, and deployment scopes have expanded. The UK AI Security Institute's "*Frontier AI Trends Report*"[16] found that in late 2023, models could only complete apprentice-level cyber tasks 9% of the time, versus 50% of the time by late 2025. Also the first model tested that could complete cyber tasks intended for experts with over ten years of experience was released in 2025.

OWASP's "*Agentic Security Initiative*" [9] identifies risks such as memory poisoning, insecure inter-agent communication, and cascading failures, reflecting patterns already being observed in early agentic deployments. Agent supply-chain threats also materialized in early 2026: Antiy CERT reported 1,184 malicious ClawHub skills in the marketplace's historical repository[23], and GitHub/NVD advisories confirmed a critical RCE vulnerability in MCP Inspector versions below 0.14.1[24].

Exploit speed has drastically changed. CrowdStrike reported the fastest observed eCrime breakout time of 27 seconds in 2025[29], emphasizing how quickly intrusions can now outrun human responses.

Question 1(e)

What unique security threats, risks, or vulnerabilities currently affect multi-agent systems, distinct from those affecting singular AI agent systems?

The OWASP *"Top 10 for Agentic Applications (2026)"*[9] includes ASI07 (Insecure Inter-Agent Communication) and ASI08 (Cascading Failures) as new vulnerability classes specific to multi-agent architectures. Multi-agent systems have cascading failure risks not present in single-agent systems (message/prompt injection between agents, emergent collusion, and cross-agent privilege escalation via delegated tasks), expanding attack surfaces beyond just one control plane.[6][1][3]

In multi-agent systems, shared memory can propagate poisoned state across agents. Shared tools and delegation channels can enable lateral movement. When agent identity, message authentication, authorization boundaries, and audit controls are weak these risks increase.[5][6][4]

Unlike single-agent failures that remain localized, multi-agent cascades propagate through feedback loops and compound into system-wide catastrophes before human operators can intervene.

Agent orchestration in multi-agent systems creates implicit trust relationships. Attackers can exploit these for lateral movement and to aggregate privileges. Compromising any single agent provides an injection vector into all downstream agents.

2. Security Practices for AI Agent Systems

Question 2(a)

What technical controls, processes, and other practices could ensure or improve the security of AI agent systems in development and deployment?

The appropriate controls, processes, and other practices best used to ensure or improve the security of AI agent systems in development and deployment depend to some extent on business use case requirements.

The following is a broad list which should be tuned to be appropriate to the business requirements and environment.

1. Plan rollback processes and procedures for an agent or the entire environment.
   a. define statistical metrics and thresholds to trigger a rollback
   b. determine required approvals and notifications
   c. define and document rollback method and procedures
   d. In critical impact applications, consider an automated rollback mechanism.

2. Implement agent containment methods or for agents with critical impact, consider an agent or system kill switch.
Determine the following:
   a. methods to implement this capability and secure it (automated for containment),
   b. metrics to monitor and thresholds required to trigger,
   c. approvals required to deploy and pre-approvals required for use of the capability,
   d. methods to ensure real-time collection and validation of required data
   e. method and channels for urgent notifications

3. Create a detailed, documented incident response plan, and documented disaster recovery plan. Test both plans before they are needed.

4. Follow a detailed, documented change control process.

5. Implement layered security controls: [2][5][4]
   a. model/interaction hardening against prompt injection;
   b. system-level containment
      i. tool allowlists,
      ii. output validation,
      iii. See also #10 - #14, below
   c. human-in-the-loop approvals for use of high risk capabilities.

6. Operationalize secure development and deployment: [15][5][6]
   a. security testing/red-teaming for agent workflows (see also #9, below)
   b. secure-by-design tool adapters,
   c. supply chain security for prompts/tools,

7. Implement continuous monitoring
   a. define and document metrics, baselines, and alert thresholds during development
      i. include anomalies in behavior and context [19][5][6]

        ii. include anomalies across multiple agents
        iii. include access, inputs and outputs of tools and agents
        iv. include metadata - IP addresses, identities, and application, session, and request IDs
    b. create policy violation alerts,
    c. create incident runbooks
    d. hash log entries for secure use
        i. analyze hashes to detect patterns of prompt abuse
        ii. store separately from raw log data
        iii. protects against passing prompt injections in log entries
    e. define strict access controls and retention policies for raw log data
        i. logs may contain privileged user or company data
        ii. logs may contain / are privileged security data

8. Incorporate AI gateways with policy engines into solutions or as shared services
    a. intercept, check, and interdict tool and agent connections.
    b. create central, immutable audit logs (see 7.a. – 7.e. above)

9. Implement ongoing evaluations of agents
    a. automate repeated testing
    b. include task specific evaluation methods
    c. iterate and improve evaluation methods
    d. NIST CAISI defensive guidance.[1]

10. Limit each identity to one agent:
    a. create a unique identity for each agent
    b. consider an identity per task per agent for high impact tasks
    c. make identity credential secrets ephemeral (changed per use)

11. Limit each tool and agent to the least required responsibilities
    a. do not aggregate tasks to a single agent except as required
    b. require human approval before consequential AI agent action
    c. do not provide agents capability to modify agent permissions
    d. split critical tasks across agents

12. Limit each tool and agent to the Least Privilege Access.
    a. grant each agent tool or scaffold identity only the minimum required permissions.
    b. limit each permission to the minimum required scope

13. Limit each tool and agent to the least required functionality:
    a. provision only the minimum required tools to each agent
    b. implement only the minimum required functionality in each tool

14. Limit each agent to the least required autonomy
    a. use the OWASP principle of “least agency”: [9]
    b. grant agents only the minimum autonomy required

15. Limit code execution by agents to sandboxed execution environments

16. Implement signing and authentication on messages between agents.

17. Do not allow unbounded access to unsecured data sources.
    a. Block or severely limit access to Internet sites if possible.
    b. Limit internal data access to purpose specific secured sources

18. Limit connectivity with AI agent system components to the least required (Zero Trust - NIST SP 800-207) [43]

19. Investigate safety risk methodologies for systemic threat modeling (example – STPA) [26]

Govern all architectural decisions by the "Limit each…" controls listed above (#10. – #14). In multi-agent systems consider the limits applied on other agents that each agent can access. Solution architectures must assume threats will be actualized and assume agents will be compromised.

Question 2(b)

To what degree, if any, could the effectiveness of technical controls, processes, and other practices vary with changes to model capability, agent scaffold software, tool use, deployment method (including internal vs. external deployment), use case, use in multi-agent systems, and otherwise?

All of the above directly impact the effectiveness and available methods to implement any given control.

Higher-capability models with broader tool access require stronger containment (sandboxing, stricter permission grants to tighter scopes, and more rigorous evaluation). [6][5][4] More capable agents can find more creative ways to circumvent constraints.

Scaffold and tool design can be leveraged to reduce dependence on self-enforced model behavioral guardrails. [2][15][5] However, controls must be re-validated when tools or other components of the scaffolding are changed, in addition to changes to models.

Use of multi-agent systems increases the complexity of logging and monitoring, managing identity and access controls, and forensics. Multi-agent systems expose a larger attack surface, including cross-agent attack paths.

A control that effectively constrains a single agent's tool access may be circumvented when agents can delegate tasks to peer agents with different permission scopes. As noted by OWASP, in multi-agent systems controls must account for propagation and amplification of faults across agents – not just individual agent containment. [28]

Existing enterprise security systems can be leveraged as a foundation for securing internal deployments. External (SaaS) deployments include less flexibility and incur dependencies on vendor controls.

Question 2(c)

How might technical controls, processes, and other practices need to change, in response to the likely future evolution of AI agent system capabilities or of the threats, risks, or vulnerabilities facing them?

Controls, processes, and practices must shift to dynamic policy enforcement of agent access and capability. Static, manual controls must be migrated to automated and integrated control systems, while at the same time care must be taken to avoid single points of failure.

Predictive behavioral modeling of agents will be increasingly important. Future controls must account for growing capability for effective deception and growing capability for agents to act on their own agency.

Supply chain will continue to grow in importance. MCP servers, for example, are living components that evolve, update, and sometimes disappear [33], and agents connect and compose them dynamically at runtime. Future supply chain controls must include signed artifacts and component specific vulnerability scanning before deployment [16].

Overconfidence in AI agents (see response to question 1(c).) increases in risk as agent capabilities and widespread agent use continue to grow. Overconfidence must be controlled by a re-coupling of deployment aspirations to deployment realities.

Question 2(d)

What are the methods, risks, and other considerations relevant for patching or updating AI agent systems throughout the lifecycle, as distinct from those affecting both traditional software systems and non-agentic AI?

AI agent patching introduces unique challenges absent from traditional software as well as from simple chatbots: in addition to updates to models, updates to scaffolding, or tool integrations can alter agent behavior in unpredictable ways. Comprehensive re-evaluation of the agent's security posture is a necessity after each update, if not also automated on a regularly scheduled basis. (see response to question 2(a).)

Uncontrolled updates from MCP servers can enable new schemas or change behavior with immediate impact, without notification or validation.[33]

Pinned tool versions and explicit upgrade approval workflows are necessary. Automated behavioral regression testing after any component update; canary deployment patterns where updated agents operate in sandboxed environments before production promotion; and immutable rollback capability to revert to the prior known-good configuration.

Question 2(e)

Which cybersecurity guidelines, frameworks, and best practices are most relevant to the security of AI agent systems?

Risk framing and lifecycle governance:
NIST AI Risk Management Framework (AI 100-1) [6] and its Generative AI Profile (AI 600-1) [4]

Secure software development practices operationalization:
NIST SP 800-218A [15] Secure Software Development Framework profile for generative AI.

Threat modeling and identification of agent-specific abuse patterns:
OWASP Top 10 for Agentic Applications (2026) [9]
MITRE ATLAS [3] - 14 agent-focused techniques added in 2025[10]

To map MITRE ATLAS threats to NIST SP 800-53 controls:
MITRE SAFE-AI framework[38]
NIST SP 800-53 Rev. 5 [5]

Agent-specific guidance is needed to adapt AC, AU, RA, and SI families.

As noted in response to question # 2(a). - Zero Trust architecture should be applied to AI agents. Both CISA and industry whitepapers suggest treating AI agents as non-trusted entities on the network.

Question 2(e)i

What is the extent of adoption by AI agent system developers and deployers of these relevant guidelines, frameworks, and best practices?

Adoption is currently limited and uneven. (see survey results in response to question 1(c))

Regulated (and thus more compliance focused) organizations may enforce existing security standards onto AI initiatives, giving partial coverage to AI agent issues.

Yet, specific AI-centric best practices (such as conducting prompt injection testing or the OWASP Top 10 for LLMs) are often only being done by early adopters and AI firms themselves.

Question 2(e)ii

What are impediments, challenges, or misconceptions about adopting these kinds of guidelines, frameworks, or best practices?

Lack of understanding of the AI and AI agent specific threats and level of risk (see response to question 1(a).) and overconfidence in AI agents and traditional security controls.

Other impediments include the novelty of agentic risks and the lack of mature agent-specific guidance and control mappings. All or nothing thinking, resource constraints and (real or perceived) lack of organizational AI security knowledge. Belief that an AI agent “isn’t risky enough” to warrant “special” controls. Belief that security controls will “stifle innovation”.

Lack of agent-specific control mappings can also lead to uncertainty about “what good looks like”. [1][6][2]

Operational friction can result from the introduction of controls. Least-privilege, least agency, and least functionality tooling, approval workflows, and continuous logging can impact latency, cost, and privacy posture, requiring design tradeoffs. [5][19][4]

Misconceptions may also lead to underinvestment in containment and monitoring. (e.g., treating agent risks as identical to chatbot risks or traditional software risk, assuming prompt hardening alone is sufficient, belief that agents pursue only objectives and methods they are given) [2][4][1]

Question 2(e)iii

Are there ways in which existing cybersecurity best practices may not be appropriate for the security of AI agent systems?

Traditional security (e.g. input validation, patches, attention to aggregate non-contextual access grants, monitoring specific to privilege elevation, etc.) does not fully address an AI agent that can be tricked via inputs or self-select harmful methods to achieve goals. Traditional best practices assume deterministic behavior defined by code and may be insufficient. Agent behavior is context-dependent and can change with prompts, memory, and upstream model updates.[6][4][1]

Perimeter-only defenses:
These are inadequate because untrusted content enters via tools that access legitimate, often internal data sources (web, email, documents). Defenses must include semantic controls and tool-level policy enforcement. [2][1][5]

Traditional perimeter security assumes human-initiated sessions – agents create persistent autonomous sessions that traverse multiple boundaries. Zero-trust architecture (NIST SP 800-207) [43] provides the closest paradigm but must be adapted for prompt injection attacks where payloads are indistinguishable from and intermingled with legitimate data.

Standard patch/vulnerability management:
"Behavioral vulnerabilities" (prompt injection susceptibility, unsafe tool chaining) require evaluation-driven remediation rather than CVE-style patching.[1][22][15]

Monitoring methodologies:
Monitoring techniques that mirror traditional software (full prompt/transcript capture) may conflict with regulations controlling PII, PHI, or financial data, without additional controls. Logging is critical. Purpose access limitations, and other governance controls are necessary alongside collection of logging data.[39][5][4] (see response to question 2(a).)

AI agents require logging of model inputs, outputs, identities, tool traces, and meta data producing large volumes of log data. Traditional one-size-fits-all retention settings need to be replaced by tiered retention, and log data must be stored separate from traditional all-in-one log storage locations.

Input sanitization:
In AI systems, simply filtering or escaping characters is not enough to prevent prompt injections. Prompt injections are natural language, no different than any other text data, with no required special characters or schema.

Identity and access management:
AI agents need to be treated as first-class identities with separate credentials. In traditional software, identities are typically created per application.

Incident response and disaster recovery:
Incident response and disaster recovery planning, including pre-approvals, are of greater importance due to the potential speed and scope of incident impacts. (see response to question 2(a).)

3. Assessing the Security of AI Agent Systems

Question 3(a)

What methods could be used during AI agent systems development to anticipate, identify, and assess security threats, risks, or vulnerabilities?

For anticipation of threats, see response to question 2(a), and implement threat modeling in the development process. In addition to traditional cybersecurity threat modeling methods, investigate use of System Theoretic Process Analysis and similar methods to identify and assess the "unknown unknowns." [26]

Question 3(a)i

What methods could be used to detect security incidents after an AI agent system has been deployed?

Logging and monitoring of all actions taken by identities used by the agents. Treat AI agents as first class identities and potential insider threats.

(see logging and monitoring in response to question 2(a).)

Monitor semantic indicators:
Repeated tool retries, unusual delegation chains, and goal drift observed across transcripts; transcript analysis methods can support scalable detection. [19][22][1]

Use "canaries" and honeytokens in high-value workflows (e.g., decoy credentials or records) to detect exfiltration attempts triggered by hijacking or unsafe tool use. [5][2][1]

Question 3(a)ii

How do these align (or differ) from traditional information security practices, including supply chain security?

Traditional information security assumes systems don't probabilistically or of their own motivation change their behavior – traditional software changes when someone changes the code.

AI models return novel responses due to novel prompts… or due to data drift, attacks which "convince" them to do something, or misalignment. Some "attacks" come through authenticated channels in the form of data ("*EchoLeak"*[8] – indirect prompt injection payload delivered in an email that passed all standard security checks).[8]

Supply chain risks in AI extend to instruction prompts, training data, parent models, and training data of parent models,

Conventional checks don't account for "data poisoning risk from a dataset supplier." Network signature checking and use of standard API gateways are irrelevant to detection of prompt injections. Virus and malware scanners are useless to detecting misalignment, or a backdoored model. These threats are invisible to traditional tools.

However, traditional information security practices do align on supply chain concerns for scaffolding and tool software and their dependencies, on change control processes, and zero

trust networking. Areas of alignment are frequently alignments of “only more so for AI agents”, including logging, least privilege, and the need for documented policies and procedures.

Red teaming overlaps with penetration testing but is primarily focused on breaking model guardrails, not probing network ports.

Logging and monitoring is traditional infosec, but for AI, organizations must additionally log prompts and outputs and tool chain traces, and statistically analyze these.

Question 3(a)iii

What is the maturity of these methods in research and applied use?

Traditional software controls that overlap with AI agents, such as IAM, zero trust networking, logging, and SDLC practices, are mature. Agent-specific tests for alignment, “hijacking”, and other novel risks, and standardized evaluation metrics are still maturing.

In research there is quite a bit more maturity for practices like red-teaming than in application.

However confounding variables for this question which are discussed elsewhere in these responses also impede the development of AI agent security methods. See responses to previous questions regarding overconfidence in AI agents - particularly questions 1(c). and 2(e)ii.

Question 3(a)iv

What resources or information would be useful for anticipating, identifying, and assessing security threats, risks, or vulnerabilities?

Assuming after deployment, as a sub-question of 3(a).

See response to question 3(b) – said documentation would be very useful.

For anticipation of threats, see relevant items in response to question 2(a), but also implement frequently scheduled automated testing (“red-team” like), post deployment, and continuous monitoring for anomalous behaviors, as discussed in response to question 2(a).

With each update or change, in addition to use of traditional cybersecurity threat modeling methods, use of System Theoretic Process Analysis or similar methods may be useful to identify and assess the “unknown unknowns.” [26]

Question 3(b)

Not all security threats, risks, or vulnerabilities are necessarily applicable to every AI agent system; how could the security of a particular AI agent system be assessed and what types of information could help with that assessment?

Thorough internal documentation points to threats, risks, and vulnerabilities of a particular AI agent system, and clues to how it should be assessed.

Documentation should include:

1. location agents will be hosted/run
2. “run-as” identities used by or available to agents,
3. capabilities available to agents including specific tool use and ability to orchestrate other agents,
    a. software used and dependencies
4. public or private network access available to agents,
5. all systems accessible to agents and permissions granted
6. all data storage accessible to agents and data classifications in these,
7. deployment location and method,
8. identities and network sources with access,
9. business use case(s),
10. all data flows and ports,
    a. all external connectivity, integration or exposure, and purpose.
    b. all internal connectivity, integration or exposure, and purpose.
11. prohibited uses and controls for these,
12. disaster recovery plan
13. incident response plan
14. solution architecture diagrams
15. source code and software component supply chain
16. maximum impact of malicious or unintended use of tools or systems accessible to agents
17. logging, monitoring, and alerting systems, processes, and procedures in use

Question 3(c)

What documentation or data from upstream developers of AI models and their associated components might aid downstream providers of AI agent systems in assessing, anticipating, and managing security threats, risks, or vulnerabilities in deployed AI agent systems?

Upstream artifacts that help downstream risk assessment of models include model/agent cards containing tool schemas, training-data source summaries, model parentage, known limitations, and model test evaluation results (e.g., robustness, hijacking susceptibility, known bias, known propensity to assist in malicious activity, etc.). [4][6][1]

AI agent system vendors should provide architecture diagrams, data flows, and component inventories for the agent stack. For agent systems deployed in the customer’s environment the components should be signed, to support supply-chain risk management. [15][5][2] Operational guidance for safe deployment should be provided including required permissions and network access, best practice configuration settings, and recovery/rollback strategies. [4][5][6]

Question 3(c)i

Does this data or documentation vary between open-source and closed-source AI models and AI agent systems, and if so, how?

Open-source models are more likely to disclose their supply chain – source of the training data, and any models trained to create the current model. Closed-source models often provide security through obscurity at the model level resulting in dependency on vendor-supplied security assurances that downstream deployers cannot independently verify.

Risk posture depends on evidence provided (evaluation results, SBOMs, integrity/signing, and incident response commitments). [15][5][6]

Question 3(c)ii

What kinds of disclosures (if made mandatory or public) could potentially create new vulnerabilities?

Publicizing a comprehensive list of prompt guardrail text or safety rules would give a roadmap to "jailbreak" an AI agent system. Threshold values used in safety systems could also become a vulnerability if disclosed. Operational details such as network endpoints or backend connections could become a blueprint for targeted attacks.

Certain disclosures should be handled through secure sharing agreements rather than public release, modeled after vulnerability disclosure guidelines in NIST SP 800-216. Transparent but secure sharing of sensitive technical specifics requires sharing via controlled channels.[5][6][15]

Disclosures must also be threat-modeled to avoid creating new attack paths or enabling evasion of monitoring controls.[6][3][5]

Question 3(c)iii

How should such, if any, disclosures be kept secure between parties to protect system integrity?

NDAs or confidential business agreements and encrypted file exchange platforms, or secure portals with strong authentication can be used to share sensitive technical details.

Question 3(d)

What is the state of practice for user-facing documentation of AI agent systems that support secure deployment?

In my experience, vendor documentation for AI agent systems is commonly fragmented, incomplete, and/or interspersed with marketing material, Vendor documentation frequently does not provide the information security professionals and users of these systems need to make informed security decisions.

Deployed AI agent system inventories and documentation needed by security professionals and IT teams supporting these systems are often scattered and incomplete. The Gravitee survey[17] found that **22.5%** of respondents' organizations have **no formal catalog** of their agents or MCP servers, while **25.4%** rely on **manual spreadsheets**, indicating that reliable asset inventory for agentic systems is often missing or immature.

4. Limiting, Modifying, and Monitoring Deployment Environments

Question 4(a)

In what manner and by what technical means could the access to or extent of an AI agent system's deployment environment be constrained?

See *"least required privilege", "least required functionality", "least required responsibility", "least required agency"* in response to question 2(a).

Also implementation of zero trust discussed in response to previous questions.

Question 4(b)

How could virtual or physical environments be modified to mitigate security threats, risks, or vulnerabilities affecting AI agent systems? What is the state of applied use in implementing undoes, rollbacks, or negations for unwanted actions or trajectories (sequences of actions) of a deployed AI agent system?

See response to question 2(a).

Virtual environments could include cloud service provider tenants or private clouds deployed in data centers.

AI specific threats, vulnerabilities, and risks are similar between virtual and physical environments except that virtual environments allow greater ease of rollback and redeployment.

Differences between these environments which are not specific to AI agents are beyond the scope of this RFI.

Question 4(c)

What is the state of managing risks associated with interactions between AI agent systems and counterparties?

Interactions with humans who are not directly using the AI system:
Risk management related to PII or PHI in data is mature.

Interactions with digital resources – services, servers, and legacy systems:
See previous responses related to overconfidence in AI agents.

Interactions with authentication mechanisms, operating system access, source code, or similar network level access vectors:
This is the highest risk area for integration of AI agent systems in most organizations, including risks from overconfidence in AI agents. Although organizations may see this as an "easy win" for AI agent use, the potential impacts of an agent security incident in this area are critical. See also responses to questions 1(a), 1(c), 2(a), and 2(e)ii.

Other AI agents:
Risk management of interactions with other agents is immature and practices are not widely deployed. AI multi-agent systems from major vendors may only have limited controls, or may disable these controls by default. [58]

Progress in this area is also likely impeded by overconfidence in AI agents - see other question responses regarding overconfidence in AI agents, and the example below.

Example:
The Gravitee survey[17] found 45.6% of teams rely on shared API keys for agent-to-agent authentication, 27.2% use custom hardcoded authorization logic. These create weak visibility into identities. When most deployed agents have weak identity and authorization boundaries, agent delegation chains of orchestrated agents can become opaque and difficult to audit. Gravitee respondents also reported 25.5% of deployed agents could orchestrate other agents.

Question 4(d)

What methods could be used to monitor deployment environments for security threats, risks, or vulnerabilities?

Implement auditing and traceability aligned to regulatory/assurance expectations where applicable (e.g., automatic logging for high-risk deployments, with highly secure log storage). [39][5][6] Monitor control effectiveness metrics over time (false positives, time-to-detect, and policy-violation rates) – continuously evaluate and manage risk. [6][5][40]

Combine traditional SIEM/SOAR with agent-specific data - structured logs of agent prompts and outputs, tool calls, identities used, and any human approvals. Add AI telemetry dashboards and consider implementing canary tests or monitor bots that continually test agents – especially orchestrator agents -- and feed results to the monitoring system. (see also response to questions 2(a)., 2(e)iii.)

Use AI agent pattern matching analysis of meta data and hashed transcripts against statistical baselines and playbooks mapped to MITRE ATLAS techniques – which can be mapped to NIST SP 800-53 controls. Use these to detect hijacking, goal drift, and anomalous behavior of delegated agents, in addition to using traditional user behavior analytics against the first class identities assigned to AI agents.

In highly critical applications, consider coupling detections to automated agent containment before human review, provided no continuity of operations conflicts. *(see related items in response to question 2(a).)*

MITRE ATLAS data is available in STIX 2.1 format for integration into security tooling. **NIST SP 800-53 AU-2, AU-3, SI-4, and IR-4** include appropriate controls for mapping to agent-specific event types, telemetry, and response workflows. MITRE SAFE-AI can be used to map these NIST SP 800-53 controls to AI threats.

Monitoring system architectures should provide tiered access levels – the scope of individuals with access should scale down dramatically as access levels go up. (see NIST SP 800-53 AU-9(4).)
Level one access - to feed analysis agents and dashboards, meta data and hashed transcript data
Level two access - strictly controlled access to anonymized log transcript data.
Level three access - after establishing high probability of a security incident, with appropriate approvals, tightly scoped, time limited access to highly privileged, non-anonymized log transcript data only if needed for forensics.

Transcript logs must be carefully guarded, possibly containing restricted or proprietary data. Also, the logs themselves can reveal AI agent application architecture, processes, and possibly system instructions, valuable intelligence for attacks or theft of trade secrets.

Question 4(d)i

What challenges exist to deploying traditional methods of monitoring threats, risks, or vulnerabilities?

Deploying traditional methods of monitoring with AI agent systems poses no special challenges. However, the results will be incomplete and monitoring ineffective due to the novel threats, risks, and vulnerabilities of AI agents, versus traditional software, which traditional methods were designed to monitor.

See responses to questions 1(a), 2(e)iii, 3(a)ii, and 4(d)

Question 4(d)ii

Are there legal and/or privacy challenges to monitoring deployment environments for security threats, risks, or vulnerabilities?

As discussed in previous responses, monitoring can create privacy and legal challenges because prompts, tool outputs, and transcripts that include these can contain sensitive personal or business data (in addition to the potential value of the logs themselves to an attacker or competitor.)

Traditional software logs contain network signals, records of logins, and other similar meta data. For effective AI agent monitoring, agent logs must contain metadata and also transcripts of prompts and responses, tool calls, and identities used. This information is “normal” natural language including between agents.

See response to question 4(d) for suggestion to address the challenge discussed in this question.

Question 4(d)iii

What is the maturity of these methods in research and practice?

Traditional logging and monitoring practices are mature, but agent-specific logging and threat detection (hijacking, tool-output injection, goal drift) are still emerging.

Standardized metrics for monitoring effectiveness are needed to compare detection approaches across agent architectures. [6][5][40]

Research tools exist (“AgentDojo” created by UK AISI Inspect Cyber) but production-grade platforms are still emerging. Major AI services companies often have in-house solutions (e.g., OpenAI monitoring for API misuse), but these are not yet widely productized.

Specialized LLM observability pipelines and structured decision-centric logging are rapidly maturing, and multi-agent threat hunting concepts show strong promise in research but lack widespread production maturity.

Question 4(e)

Are current AI agent systems widely deployed on the open internet, or in otherwise unbounded environments?

Commercially available AI agent systems commonly include open access to the Internet by AI agents though not AI agents deployed directly exposed to the Internet.

I have regularly seen this in products I have researched in my role as a Principal Security Architect.

5. Additional Considerations

Question 5(a)

What methods, guidelines, resources, information, or tools would aid the AI ecosystem in the rapid adoption of security practices affecting AI agent systems and promoting the ecosystem of AI agent system security innovation?

Create a curated threat intelligence and known exploit sharing mechanism. Publish reference architectures, control mappings, and evaluation playbooks for agent systems (hijacking tests, tool safety, monitoring patterns) to accelerate consistent adoption. Incorporate these into a tiered maturity model and sector specific implementation guidelines.

Extend NIST AI 600-1 with control-by-control implementation guidance for agent deployments to provide organizations with a concrete compliance roadmap.

Map existing frameworks to agent-specific controls. Build on the MITRE SAFE-AI Framework which already maps to and describes AI related residual risks in NIST SP 800-53.[38] Consider promoting this as a community-driven framework effort modeled on the OWASP GenAI Security Project. [30]

Question 5(b)

In which policy or practice areas is government collaboration with the AI ecosystem most urgent or most likely to lead to improvements in the state of security of AI agent systems today and into the future?

Public-private information sharing on threats and mitigations can improve defensive posture without overexposing sensitive details. [5][6][15] Regulatory clarity on logging/record-keeping and cybersecurity expectations for high-risk systems can reduce uncertainty and incentivize secure deployments. [45][39][5]

The MITRE Secure AI Program, supported by 16 member organizations, provides a model for incident sharing, but government support is needed to accelerate adoption across sectors

Question 5(c)

In which critical areas should research be focused to improve the current state of security practices affecting AI agent systems?

Suggested areas for research focus on improving the current state of security practices for AI agents:

1. Effective education to combat human overconfidence in AI agents.
2. Application of threat modeling and risk management methodologies from outside traditional cybersecurity.
3. Development of a security control catalog focused on AI agent systems

Question 5(c)i

Where should future research be directed in order to unlock the benefits of adoption of secure and resilient AI agent systems?

Unlocking the benefits of adoption of secure and resilient AI agent systems requires mitigating the risks of human overconfidence – see also previous question. I believe this is the most critical need, today.

Future research should also emphasize practical implementation guidelines for deployments:

1. Patterns for safe tool use, bounded privileges, and effective monitoring in complex environments.
2. Effective, repeatable patterns for AI governance that promotes AI use without introducing new operational risk.
3. Clear guidance for measurement approaches to quantify security posture and control effectiveness over time.[6][40][39]

Question 5(c)ii

Which research approaches should be prioritized to advance the scientific understanding and mitigation of security threats, risks, and vulnerabilities affecting AI agent systems?

Research on detection of misaligned or deceptive agent behavior - reliable detection of and controls for specification gaming, deceptive alignment, and covert goal pursuit in deployed agents should be critical priorities for national security.

Additional suggestions:
Combine sandbox research studies with collected real world telemetry (tool gateway logs and transcript analysis) to validate that research results hold under real deployment conditions.

Question 5(d)

How are other countries addressing these challenges and what are the benefits and drawbacks of their approaches?

The European Union has adopted the most comprehensive and binding approach via the EU AI Act, which applies from 2 August 2026, with some provisions applying earlier or later. The EU AI Act mandates risk management frameworks, cybersecurity protections, human oversight, and transparency requirements, with fines up to EUR35 million or 7% of global turnover.

Benefits: creates legal certainty and strong compliance incentives.
Drawbacks: may constrain innovation velocity, and the Act does not yet specifically address agentic AI architectures.

The United Kingdom has pursued an evaluation-led approach through the UK AI Security Institute, conducting pre-deployment evaluations of frontier models and publishing the Frontier AI Trends Report [16], but has not adopted binding AI-specific legislation.
Benefits: enables rapid, evidence-based response.
Drawbacks: lacks enforcement mechanisms.

China has pursued a dual-track strategy integrating AI via state planning while managing social risks through specific regulations targeting deepfakes and generative AI content.

The UK and US AI Safety Institutes signed a landmark agreement to jointly test advanced AI models.

Question 5(e)

Are there practices, norms, or empirical insights from fields outside of artificial intelligence and cybersecurity that might benefit our understanding or assessments of the security of AI agent systems?

Systems safety engineering:
The key insight from safety engineering is that safety must be enforced by systems independent of the process they control – a principle that mandates external policy engines and monitoring systems rather than relying on the agent's own alignment. Additionally, the aviation concept of "sterile cockpit" rules maps to restricting agent autonomy during high-consequence operations. Financial market circuit breakers provide precedent for automated halt mechanisms when anomalous behavior is detected.

Systems safety engineering methods (e.g., STPA) provide hazard/constraint-focused analysis that can complement cybersecurity threat models for complex, software-intensive, autonomous systems. [26][6][5] STPA, developed by Nancy Leveson at MIT (*Engineering a Safer World*, MIT Press, 2011), provides a rigorous methodology for identifying unsafe control actions in hierarchical control structures – directly applicable to the principal-agent-tool hierarchy in agentic systems.

Research on alarm fatigue in clinical settings is relevant to designing agent monitoring dashboards.

The concept of "meaningful human control" from autonomous weapons governance provides a framework for defining what constitutes adequate oversight of consequential agent actions.

References

[1] NIST CAISI "Strengthening AI Agent Hijacking Evaluations" Technical Blog, January 2025. https://www.nist.gov/news-events/news/2025/01/technical-blog-strengthening-ai-agent-hijacking-evaluations
[2] OWASP "Top 10 for Large Language Model Applications." https://owasp.org/www-project-top-10-for-large-language-model-applications/
[3] MITRE ATLAS (Adversarial Threat Landscape for Artificial-Intelligence Systems). https://atlas.mitre.org/
[4] NIST "Artificial Intelligence Risk Management Framework: Generative Artificial Intelligence Profile" (NIST AI 600-1), July 2024. https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf
[5] NIST "Security and Privacy Controls for Information Systems and Organizations" SP 800-53 Rev. 5, Update 1, December 2024. https://csrc.nist.gov/pubs/sp/800/53/r5/upd1/final
[6] NIST "Artificial Intelligence Risk Management Framework" (AI RMF 1.0), NIST AI 100-1, January 2023. https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-1.pdf
[7] NIST "Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations" NIST AI 100-2e2025, March 2025. https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.100-2e2025.pdf
[8] CATO Networks, "Breaking down 'EchoLeak', the First Zero-Click AI Vulnerability Enabling Data Exfiltration from Microsoft 365 Copilot" May 31, 2025. https://www.catonetworks.com/blog/breaking-down-echoleak/
[9] OWASP "Top 10 for Agentic Applications (2026)" GenAI Security Project, December 10, 2025. https://genai.owasp.org/resource/owasp-top-10-for-agentic-applications-for-2026/
[10] Zenity Labs, "Zenity Labs and MITRE ATLAS Collaborate to Advance AI Agent Security with the First Release of 14 Agent-Focused Techniques," October 2025. https://zenity.io/blog/current-events/zenity-labs-and-mitre-atlas-collaborate-to-advances-ai-agent-security-with-the-first-release-of
[11] RadWare, "ShadowLeak: The First Service-Side Leaking, Zero-click Indirect Prompt Injection Vulnerability", 2025. https://www.radware.com/security/threat-advisories-and-attack-reports/shadowleak/
{12 – 14 intentionally removed in editing}
[15] NIST "Secure Software Development Practices for Generative AI and Dual-Use Foundation Models: An SSDF Community Profile" SP 800-218A, June 2024. https://csrc.nist.gov/pubs/sp/800/218/a/final
[16] UK AI Security Institute (AISI), "Frontier AI Trends Report," December 18, 2025. https://www.aisi.gov.uk/frontier-ai-trends-report
[17] Gravitee, "The State of AI Agent Security 2026" Survey Report (919 respondents), February 2026. https://www.gravitee.io/state-of-ai-agent-security
{18 intentionally removed in editing}
[19] NIST CAISI Research Blog, "Analyzing Transcripts from AI Agent Evaluations," February 18, 2026. https://www.nist.gov/blogs/caisi-research-blog/analyzing-transcripts-ai-agentevaluations
[20] Zapier, "State of Agentic AI Adoption Survey" (525 US C-Suite respondents), December 15, 2025. https://zapier.com/blog/ai-agents-survey/
{21 intentionally removed in editing}
[22] UK AI Security Institute, Inspect Evals (agent evaluations). https://inspect.aisi.org.uk/evals/
[23] Antiy Labs, "ClawHavoc: Analysis of Large-Scale Poisoning Campaign Targeting the OpenClaw Skill Market for AI Agents" February 2026. https://www.antiy.net/p/clawhavoc-analysis-of-large-scale-poisoning-campaign-targeting-the-openclaw-skill-market-for-ai-agents/
[24] NIST NVD, "CVE-2026-23744 Detail" January 16, 2026. https://nvd.nist.gov/vuln/detail/CVE-2026-23744
[25] ISO/IEC 42001:2023, "Artificial Intelligence — Management System." https://www.iso.org/standard/42001

[26] Leveson, N. & Thomas, J., "STPA Handbook." https://www.flighttestsafety.org/images/STPA_Handbook.pdf
{27 intentionally removed in editing}
[28] OWASP Top 10 for Agentic Applications for 2026," ASI08: Cascading Failures", 2026. https://genai.owasp.org/download/52117/?tmstv=1765059207
[29] CrowdStrike, "CrowdStrike 2026 Global Threat Report", 2026. https://www.crowdstrike.com/en-us/global-threat-report/
[30] OWASP GenAI Security Project. https://genai.owasp.org/
{31 - 32 intentionally removed in editing}
[33] Dave Patten, "MCP's Next Phase: Inside the November 2025 Specification," Medium, November 2025. https://medium.com/@dave-patten/mcps-next-phase-inside-the-november-2025-specification-49f298502b03
[34] NIST CAISI, "Cheating on AI Agent Evaluations," November 2025. https://www.nist.gov/caisi/cheating-ai-agent-evaluations
{35 - 37 intentionally removed in editing}
[38] MITRE, "SAFE-AI: Situational Awareness Framework for Evaluation of AI," 2025. https://atlas.mitre.org/pdf-files/SAFEAI_Full_Report.pdf
[39] European Commission (AI Act Service Desk), "Article 12: Record-keeping." https://ai-act-service-desk.ec.europa.eu/en/ai-act/article-12
[40] UK Government, "AI Safety Institute Approach to Evaluations," February 9, 2024. https://www.gov.uk/government/publications/ai-safety-institute-approach-to-evaluations/aisafety-institute-approach-to-evaluations
{41 - 42 intentionally removed in editing}
[43] NIST, "Zero Trust Architecture" SP 800-207, August 2020. https://csrc.nist.gov/pubs/sp/800/207/final
{44 intentionally removed in editing}
[45] European Commission (AI Act Service Desk), "Article 15: Accuracy, Robustness and Cybersecurity." https://ai-act-service-desk.ec.europa.eu/en/ai-act/article-15
{46 intentionally removed in editing}
[47] Donghyun Lee, Mo Tiwari, "Prompt Infection : LLM-to-LLM Prompt Injection within Mult-Agent Systems", October, 2024. https://arxiv.org/abs/2410.07283
[48] Hung, Kuo-Han, et al., "\attn: Detecting Prompt Injection Attacks in LLMs", April 23, 2025. https://arxiv.org/html/2411.00348v2
[49] Russell, Stuart J. and Norvig, Peter, "Artificial Intelligence: A Modern Approach", 1st edition, 1995. https://people.eecs.berkeley.edu/~russell/intro.html
[50] Vaswani, Ashish et al., "Attention is all you need", June 12, 2017. https://arxiv.org/abs/1706.03762
[51] Shapira, Natalie et al., "Agents of Chaos", February 23, 2026. https://arxiv.org/pdf/2602.20021
[52] PC Magazine, "Vibe Coding Fiasco: AI Agent Goes Rogue, Deletes Company's Entire Database", July 2025. https://www.pcmag.com/news/vibe-coding-fiasco-replite-ai-agent-goes-rogue-deletes-company-database
[53] Anthropic, "Agentic misalignment: How LLMs could be insider threats", June 20, 2025. https://www.anthropic.com/research/agentic-misalignment
{54 - 55 intentionally removed in editing}
[56] Hubbinger, Evan et al., "Sleeper Agents: Training Deceptive LLMs that Persist Through Safety Training", January 2024. https://arxiv.org/html/2401.05566
[57] Koorndijk, Jeanice, "Empirical Evidence for Alignment Faking in a Small LLM and Prompt-Based Mitigation Techniques", June 2025. https://arxiv.org/html/2506.21584
[58] Microsoft, "**Orchestrate agent behavior with generative AI**", February 2026. https://learn.microsoft.com/en-us/microsoft-copilot-studio/advanced-generative-actions?source=recommendations